

# StaySignal — Hotel Booking Cancellation Prediction



Python

3.10+



API

FastAPI



ML

XGBoost



Container

Docker

StaySignal estimates the likelihood that a hotel reservation will be cancelled before arrival. It combines a trained classification pipeline with booking-level domain rules and exposes predictions through a FastAPI service and a browser-based interface. The goal is to help hotel teams identify potentially risky bookings earlier and make more informed planning and revenue decisions.

**Architecture image note:** The supplied image is included at [assets/staysignal-architecture.png](#). It illustrates the system at a glance; details in this README are checked against the repository code. In particular, the repository contains a Dockerfile but does **not** contain a Railway deployment configuration or deployment proof, and its training script compares fixed model configurations rather than implementing an automated hyperparameter-search process.

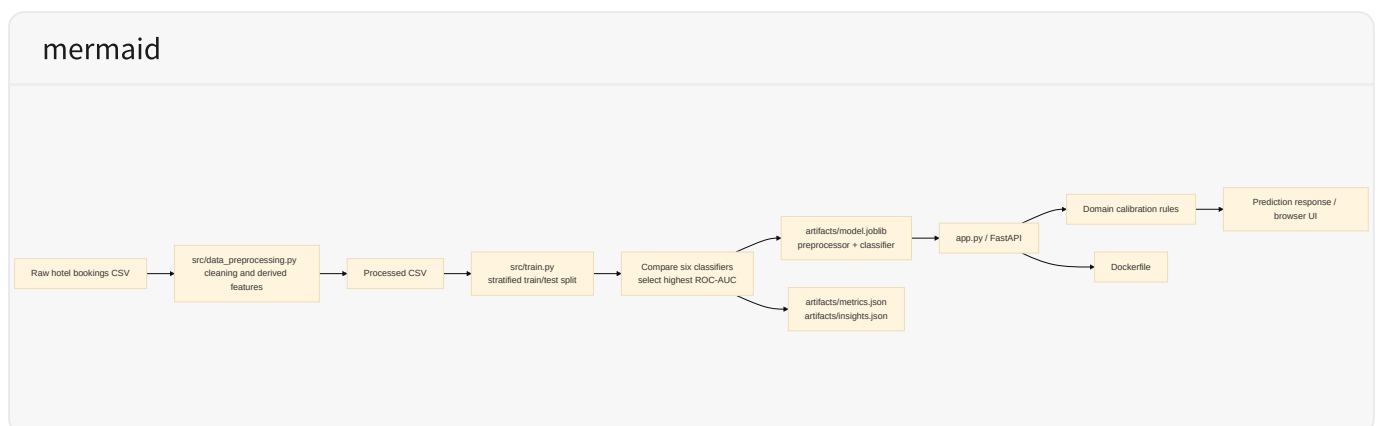
## Problem Statement

Unexpected booking cancellations can leave rooms unoccupied, disrupt staffing and inventory plans, and reduce expected revenue. A useful signal before arrival gives teams more time to review at-risk reservations and consider appropriate operational responses, such as adjusting room plans or prioritizing follow-up.

StaySignal predicts the dataset target `is_canceled` from reservation attributes. The prediction is a risk estimate, not a guarantee that a guest will cancel, and should support—not replace—hotel policy and staff judgment.

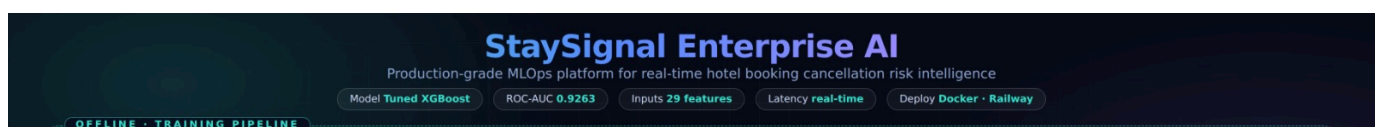
## Solution Overview

The repository implements a data-cleaning script, a model-comparison training script, a saved scikit-learn pipeline artifact, a FastAPI inference application, and a Dockerfile. The trained pipeline is loaded by the API at process startup.



The training script selects the candidate with the highest ROC-AUC on its held-out test split. It evaluates XGBoost, Random Forest, Gradient Boosting, Decision Tree, AdaBoost, and Logistic Regression; the checked-in metrics identify **Tuned XGBoost** as the selected model.

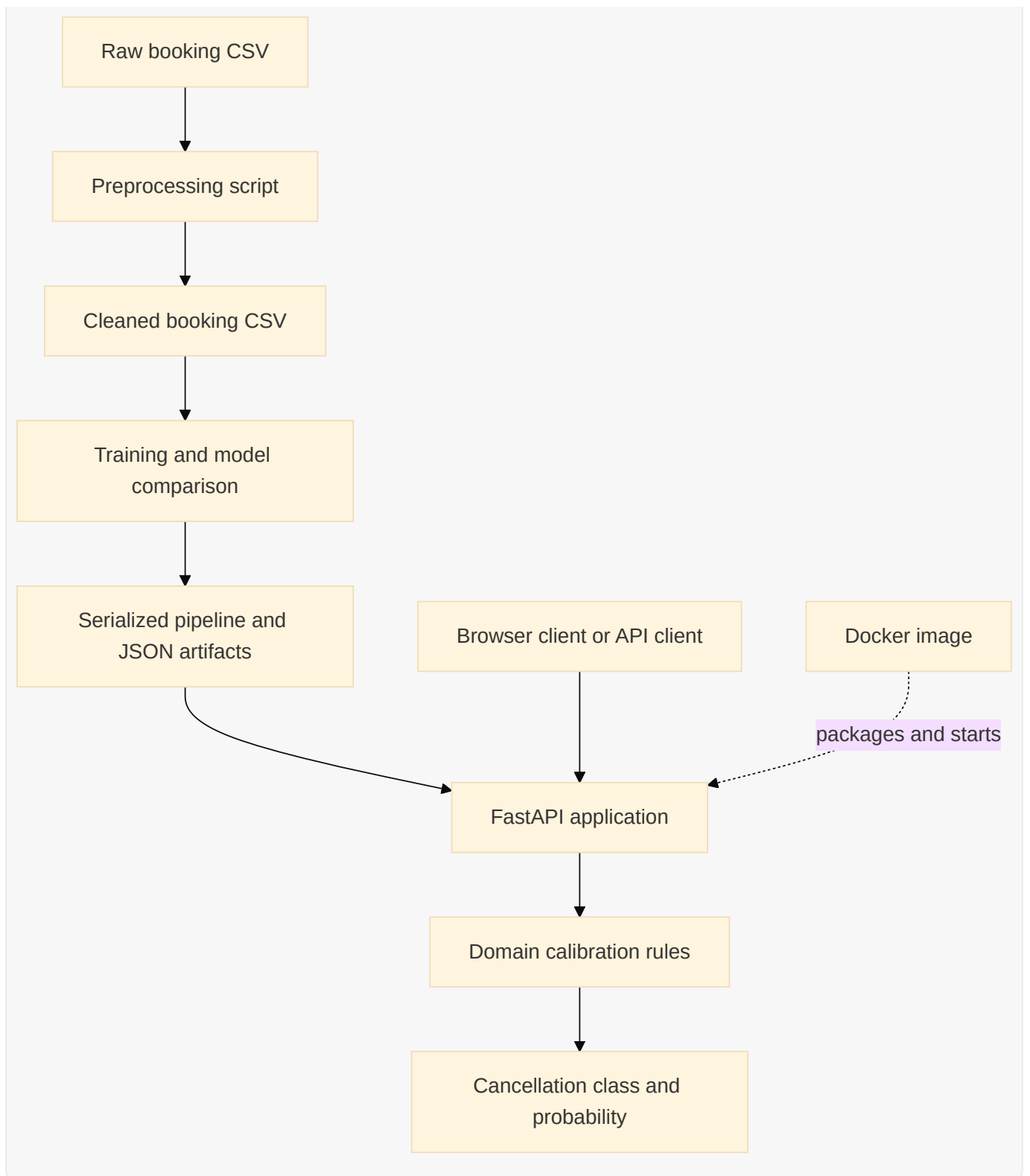
## System Architecture





The image above is the architecture illustration supplied for this README. The code-level flow is summarized here:

mermaid



`app.py` loads `artifacts/model.joblib`, serves the static frontend, provides health and model-metric routes, and handles prediction requests. The prediction endpoint first obtains a model probability and then applies explicit booking-domain adjustments before returning the final probability and thresholded class. Docker packages the application and starts Uvicorn; the repository does not include a cloud deployment configuration.

## ML Pipeline

1. `src/data_preprocessing.py` loads `data/raw/hotel_bookings.csv` , removes duplicate rows, fills selected missing values, derives `total_stays` and `total_guests` , and removes records with zero total guests. It writes `data/processed/cleaned_hotel_bookings.csv` .
2. `src/train.py` reads that processed file and uses `is_canceled` as the target. It excludes `reservation_status` and `reservation_status_date` from the predictors.
3. The data is split into training and test sets using an 80/20 split, `random_state=42` , and stratification by target.
4. The pipeline standardizes numeric columns and one-hot encodes categorical columns ( `handle_unknown='ignore'` ). Six classifiers are fitted and evaluated.
5. The model with the greatest test-split ROC-AUC is saved as `artifacts/model.joblib` , together with its metrics and dataset summary JSON files.
6. At inference, `app.py` creates the derived features, sends the booking through the serialized pipeline, applies the domain-calibration rules, clips the result to the configured range, and returns JSON.

## How a Prediction Works

1. A client submits booking data to `POST /predict` .
2. FastAPI validates/coerces the request against the `BookingData` Pydantic model. Every field has a default, so requests may omit fields; the example below supplies them explicitly.
3. The API normalizes recognized `deposit_type` values and derives `total_stays` and `total_guests` .
4. The serialized pipeline applies its fitted numeric and categorical transformations and computes the base cancellation probability.
5. Domain rules adjust that base probability using selected booking signals.
6. The adjusted probability is clipped to **0.03–0.96**. A final probability of at least **0.50** maps to `is_canceled_prediction: 1` ; otherwise it is `0` .
7. The API returns the class, final probability, and a success status.

## Feature Engineering and Inputs

The preprocessing function creates two derived features:

| Feature | Construction | Why it can help |
|---------|--------------|-----------------|
|---------|--------------|-----------------|

|                           |                              |                                                                             |
|---------------------------|------------------------------|-----------------------------------------------------------------------------|
| <code>total_stays</code>  | Weekend nights + week nights | Represents the full planned length of stay.                                 |
| <code>total_guests</code> | Adults + children + babies   | Represents the total party size and supports removal of zero-guest records. |

The model also uses the reservation fields retained in the cleaned dataset, including lead time, arrival date, hotel, guest counts, market/channel, prior booking history, room types, deposit type, booking changes, waiting-list days, rate, parking spaces, and special requests. Numeric fields are standardized and categorical fields are one-hot encoded by the saved pipeline.

Selected data cleaning rules fill missing `company` , `agent` , and `children` values with `0` , fill missing `country` with `Unknown` , remove duplicates, and discard records whose derived `total_guests` is zero. The serving code additionally normalizes several deposit labels to `Non Refund` , `Refundable` , or `No Deposit` ; it also scales `required_car_parking_spaces` by `0.15` in the prediction input before passing it to the pipeline.

## Model and Recorded Evaluation

This is a **binary classification** problem. The target is `is_canceled` ; the predictors are the processed reservation fields after removal of the target and the two reservation-outcome fields noted above. The training script compares six classifiers and uses ROC-AUC to select the saved candidate. For the XGBoost candidate, the code sets 300 estimators, a 0.05 learning rate, depth 10, row/column subsampling of 0.8, and a positive-class weight derived from the training split.

The committed `artifacts/metrics.json` reports the following values for the selected model:

| Metric    | Recorded value |
|-----------|----------------|
| Accuracy  | 0.8433         |
| Precision | 0.6663         |
| Recall    | 0.8632         |
| F1-score  | 0.7521         |
| ROC-AUC   | 0.9263         |

These are the metrics stored with the repository artifact; they are not a guarantee of future performance. The training code calculates metrics on its held-out test split before the serving-time domain calibration layer. The calibration rules are not separately evaluated in the training script.

The serialized `artifacts/model.joblib` contains the preprocessing-and-classifier pipeline, so inference uses the fitted transforms together with the selected estimator. The repository also contains `artifacts/insights.json`, with summary statistics for the processed data.

## Domain Calibration Layer

The API distinguishes the model's **base probability** from the **final cancellation probability**:

- **Base probability:** `predict_proba` output from the saved fitted pipeline.
- **Final probability:** Base probability plus the serving code's hand-authored calibration adjustment, clipped to the interval from 0.03 to 0.96.

The rules adjust risk based on lead time, previous cancellations, booking changes, special requests, parking, deposit type, and repeat-guest status. For example, long lead times and prior cancellations can increase the adjustment, while booking changes, special requests, parking, refundable deposits, and repeat-guest status can reduce it under the implemented conditions. This is deterministic business/domain logic applied after the base model—not a second trained model or a probability-calibration model fitted on a calibration dataset.

## FastAPI

The application is defined in `app.py`. The API description and interactive OpenAPI documentation are available at `/docs` when the service is running.

| Method | Endpoint             | Purpose                                                                                              | Response                                                                                  |
|--------|----------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| GET    | <code>/</code>       | Serves <code>static/index.html</code> when present; otherwise returns a small service-status object. | HTML page or JSON fallback.                                                               |
| GET    | <code>/health</code> | Reports service health and whether the model loaded.                                                 | <code>{"status":"healthy","model_loaded":true}</code> (the Boolean reflects the runtime). |
| GET    | <code>/models</code> | Returns <code>artifacts/metrics.json</code>                                                          | The saved model name and metrics                                                          |

|      |                       |                                                 |                                                  |
|------|-----------------------|-------------------------------------------------|--------------------------------------------------|
|      |                       | when present.                                   | JSON.                                            |
| POST | /predict              | Scores a booking and applies domain rules.      | Prediction class, final probability, and status. |
| GET  | /static/...           | Serves files mounted from static/ .             | Static frontend assets.                          |
| GET  | /style.css , /main.js | Serves the corresponding static frontend files. | CSS or JavaScript.                               |

## Prediction request

All fields below are defined in the request schema. The schema assigns defaults to each field; Pydantic validates/coerces their declared types, but the code does not define additional numeric range constraints.

JSON

```
{
  "hotel": "City Hotel",
  "lead_time": 60,
  "arrival_date_year": 2026,
  "arrival_date_month": "October",
  "arrival_date_week_number": 41,
  "arrival_date_day_of_month": 8,
  "stays_in_weekend_nights": 2,
  "stays_in_week_nights": 3,
  "adults": 2,
  "children": 0.0,
  "babies": 0,
  "meal": "BB",
  "country": "PRT",
  "market_segment": "Online TA",
  "distribution_channel": "TA/TO",
  "is_repeated_guest": 0,
  "previous_cancellations": 0,
  "previous_bookings_not_canceled": 0,
  "reserved_room_type": "A",
  "assigned_room_type": "A",
  "booking_changes": 0,
  "deposit_type": "No Deposit",
  "agent": 9.0,
  "company": 0.0,
  "days_in_waiting_list": 0,
```

```
"customer_type": "Transient",
"adr": 120.0,
"required_car_parking_spaces": 0,
"total_of_special_requests": 0
}
```

Successful response shape:

JSON

```
{
  "is_canceled_prediction": 0,
  "cancellation_probability": 0.1234,
  "status": "success"
}
```

The probability shown above is illustrative of the response format; actual values depend on the request and loaded artifact. If the model is not loaded, `/predict` returns HTTP 500. Errors during prediction are returned as HTTP 400 with a detail message.

## Local Setup

The repository's `requirements.txt` is stored as UTF-16, rather than the UTF-8 text commonly expected by pip. Convert it to UTF-8 before installing dependencies or building the Docker image:

Bash

```
git clone https://github.com/domaamohamed00-ui/StaySignal-Enterprise-AI.git
cd StaySignal-Enterprise-AI

python -c "from pathlib import Path; p=Path('requirements.txt' ); p.write_text(

python -m venv .venv
source .venv/bin/activate          # Windows PowerShell: .venv\Scripts\Activate.p
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The checked-in API startup command is Uvicorn:

Bash

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Open `http://localhost:8000/` for the web interface or `http://localhost:8000/docs` for the interactive API documentation. The app expects `artifacts/model.joblib` and, for model metrics, `artifacts/metrics.json` to be available relative to the working directory; both are included in the repository.

## Re-run preprocessing and training

From the repository root, after installing dependencies:

Bash

```
python src/data_preprocessing.py
python src/train.py
```

The scripts use repository-relative input/output paths. Training writes the selected pipeline and JSON artifacts under `artifacts/`. It compares all six model candidates each time it runs.

## API Usage

With the API running locally, send a request with `curl` :

Bash

```
curl -X POST "http://localhost:8000/predict" \
-H "Content-Type: application/json" \
-d '{
  "hotel": "City Hotel",
  "lead_time": 60,
  "arrival_date_year": 2026,
  "arrival_date_month": "October",
  "arrival_date_week_number": 41,
  "arrival_date_day_of_month": 8,
  "stays_in_weekend_nights": 2,
  "stays_in_week_nights": 3,
  "adults": 2,
  "children": 0,
  "babies": 0,
  "meal": "BB",
  "country": "PRT",
  "market_segment": "Online TA",
  "distribution_channel": "TA/TO",
  "is_repeated_guest": 0,
  "previous_cancellations": 0,
  "previous_bookings_not_canceled": 0,
  "reserved_room_type": "A",
  "assigned_room_type": "A",
```

```

    "booking_changes": 0,
    "deposit_type": "No Deposit",
    "agent": 9,
    "company": 0,
    "days_in_waiting_list": 0,
    "customer_type": "Transient",
    "adr": 120,
    "required_car_parking_spaces": 0,
    "total_of_special_requests": 0
  }'

```

A successful call returns a JSON object containing `is_canceled_prediction` (0 or 1), `cancellation_probability` (the final clipped probability), and `status` ( `success` ).

## Docker

The Dockerfile uses `python:3.10-slim` , installs `requirements.txt` , copies the repository into `/app` , and starts Uvicorn on `0.0.0.0` . The container port defaults to 8000 and can be overridden with the `PORT` environment variable.

Because the checked-in requirements file is UTF-16, convert it before building:

Bash

```

python -c "from pathlib import Path; p=Path('requirements.txt'); p.write_text(p.read_text(encoding='utf-16'), encoding='utf-8')"
docker build -t stayssignal .
docker run --rm -p 8000:8000 -e PORT=8000 stayssignal

```

Then visit `http://localhost:8000/` . The image packages the API, static frontend, model artifact, and repository files copied by the Dockerfile. It provides a repeatable base runtime, but the project does not include deployment orchestration or a cloud deployment target.

## Project Structure

Plain Text

```

StaySignal-Enterprise-AI/
├── app.py                # FastAPI app, request schema, inference logic
├── Dockerfile            # Python 3.10 image and Uvicorn startup
├── requirements.txt      # Python dependencies (currently UTF-16 encoded)
├── assets/
│   └── stayssignal-architecture.png    # Supplied system-architecture illustration
├── artifacts/
│   ├── insights.json    # Dataset summary statistics
│   └── metrics.json     # Selected model's stored evaluation metrics

```

```

├── model.joblib                # Serialized preprocessing/model pipeline
├── data/
│   ├── raw/hotel_bookings.csv  # Raw booking dataset (119,390 rows)
│   └── processed/cleaned_hotel_bookings.csv # Processed dataset (87,230 rows)
├── notebooks/
│   └── 01_eda.ipynb            # Empty notebook in the checked-in snapshot
├── src/
│   ├── data_preprocessing.py    # Duplicate/missing-value handling and
│   └── train.py                 # Train, compare, evaluate, and export
└── static/
    ├── index.html              # Browser interface
    ├── main.js                  # Frontend behavior and API calls
    └── style.css                # Frontend styles

```

The notebook currently has no content.

## MLOps Scope

### Implemented in this repository

- Data cleaning and feature creation in a preprocessing script.
- A stratified train/test split and comparison of six classifiers.
- Numeric scaling and categorical one-hot encoding inside the fitted pipeline.
- Selection and serialization of the best candidate by test-split ROC-AUC.
- JSON artifacts for model metrics and dataset insights.
- FastAPI prediction, health, and metrics routes, plus a static browser interface.
- A Dockerfile for containerized API startup.

### Not currently implemented

The repository does not include experiment tracking, a model registry, automated CI/CD, production telemetry/monitoring, data or concept drift detection, automated retraining, or cloud deployment configuration. These are potential future MLOps improvements, not current capabilities.

## Business Impact

A cancellation-risk estimate can help hotel teams prioritize attention toward bookings that may warrant review, improve room and staffing plans, and make revenue-management decisions with more lead time. The repository demonstrates a technical prediction workflow; it does not measure realized revenue gains or prescribe automatic interventions.

## Future Improvements

- Add reproducible dependency/version pinning and automated tests for preprocessing and API behavior.
- Add experiment tracking and a versioned model registry.
- Introduce CI/CD checks and a repeatable deployment workflow.
- Monitor API health, prediction quality, and model performance over time.
- Add data-quality and drift monitoring, with a governed retraining process.
- Keep the architecture illustration synchronized with implemented code and deployment configuration.
- Evaluate and document the effect of domain-calibration rules against an appropriate validation set.
- Deploy to a cloud environment with configuration and operational controls.

## Technologies

| Area               | Verified technologies                                                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------|
| Language and data  | Python, Pandas, NumPy                                                                                               |
| Machine learning   | scikit-learn, XGBoost, joblib                                                                                       |
| API and validation | FastAPI, Uvicorn, Pydantic                                                                                          |
| Packaging          | Docker                                                                                                              |
| Frontend assets    | HTML, CSS, JavaScript; frontend references Bootstrap, Font Awesome, Google Fonts, Chart.js, and html2pdf.js via CDN |

## Project Status

**Core model training, saved-artifact inference, a FastAPI service, a browser interface, and Docker packaging are present.** The checked-in project does not provide a cloud deployment, production monitoring, or the broader MLOps lifecycle capabilities listed as future improvements. The requirements file encoding should be corrected in the repository to make installation and Docker builds straightforward.

# Author

Created by  [@domaamohamed00-ui](#).